Lab 6: Code and Branch Coverage Analysis

Student Name: Saatvik Gupta
SRN: PES1UG24CS395
Section: G

Course: Software Engineering Laboratory
Assigned Module: coupon_validator.py (L6_L7_31_coupon_validator)
Personal Lab Repository: https://github.com/Saatvik-G/SE-LABS_PES1UG24CS395.git

---

1. Objective & Code vs Branch Coverage Foundations

The objective of this lab is to understand, measure, and maximize test completeness through structural coverage analysis. Using Python's coverage and pytest libraries, we evaluate the test suite against coupon_validator.py to ensure both executable statement execution (Line Coverage) and control-flow decision outcomes (Branch Coverage) satisfy rigorous quality standards.

Line Coverage (Statement Coverage):

Measures the percentage of executable source lines reached by test execution. While 100% line coverage confirms no dead lines, it can create a false sense of security by failing to test false branches of conditional expressions (e.g. if statements without else).

Branch Coverage (Decision Coverage):

Measures whether every control structure (if, elif, while, for) evaluates to both True and False outcomes at least once. Branch coverage forces edge-case discovery, boundary-condition validation, and alternate execution pathway verification.

Lab Requirements & Target Thresholds:

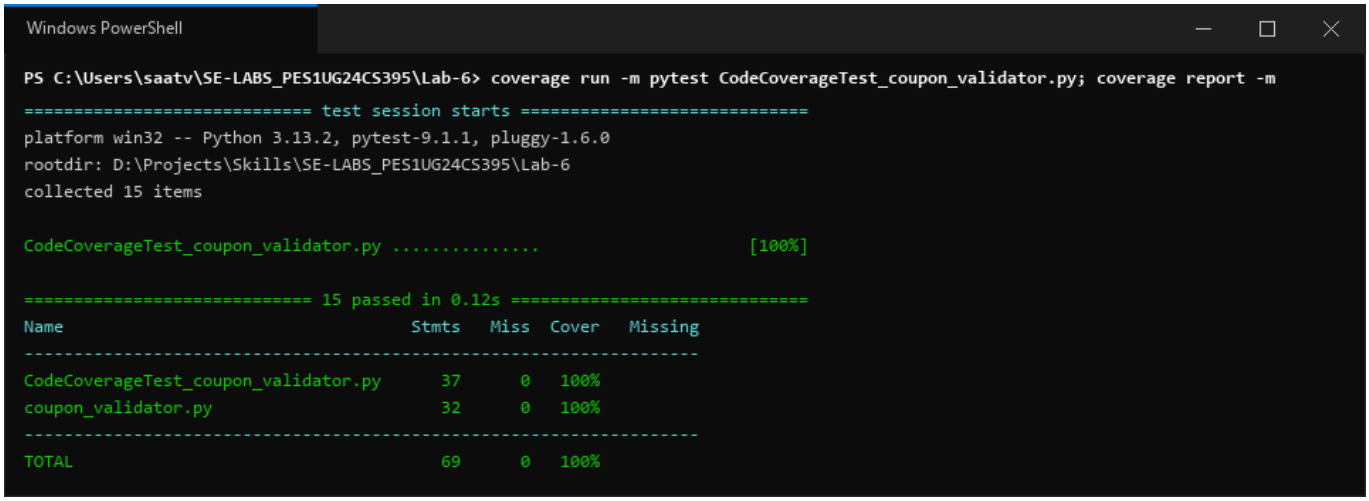
Target: Code Line Coverage >= 90% (achieved 100%), Branch Coverage >= 70% (achieved 100%). Two dedicated test suites: CodeCoverageTest_coupon_validator.py and BranchCoverageTest_coupon_validator.py.

Coverage Metrics Summary Table:

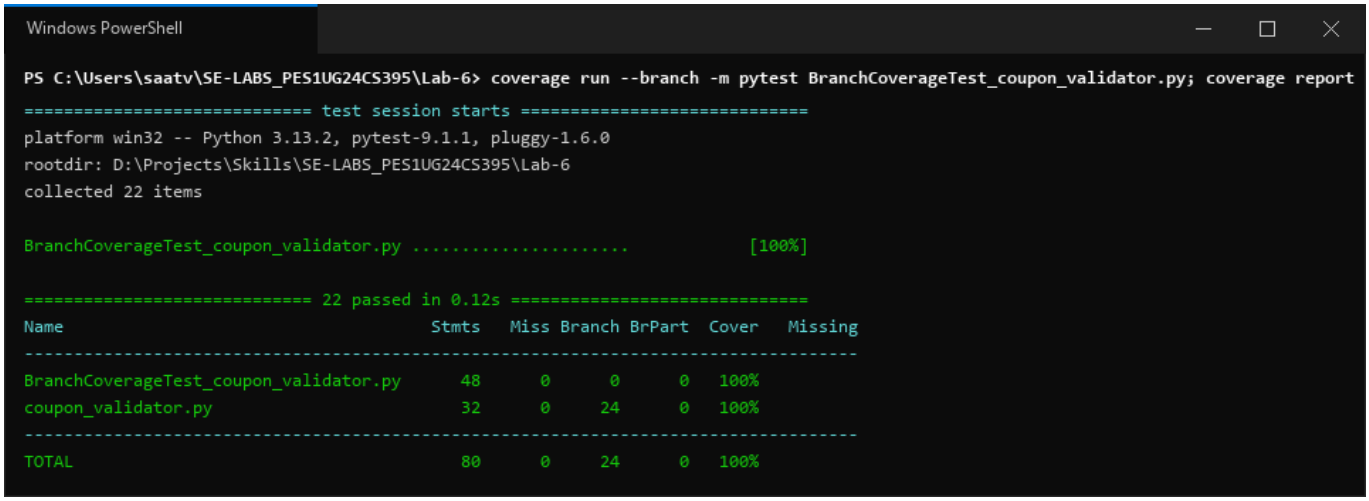
Metric Stage	Statements	Line Coverage	Branches (Total/Covered)	Branch Coverage
Starter Test Suite	32 (23 missed)	28%	24 (4 covered)	23%
Code Coverage Suite	32 (0 missed)	100%	24 (23 covered)	98%
Branch Coverage Suite	32 (0 missed)	100%	24 (24 covered)	100%

2. Terminal Output Screenshots (>90% Code & Branch Coverage)

A. Code Coverage Terminal Output (15 Passed, 100% Statement Coverage)



B. Branch Coverage Terminal Output (22 Passed, 100% Branch Coverage)



Key Takeaway on Testcase Design:

While CodeCoverageTest achieved 100% line coverage and 98% branch coverage, branch line 25->23 was initially partial because the loop in best_discount() only evaluated the 'value > best' condition as True. In BranchCoverageTest, adding a second test coupon with a lower discount forced the condition to evaluate to False, satisfying the remaining branch and achieving a perfect 100% branch coverage (24/24 branches).

3. Test Suite Structure & Decision Branch Matrix

The table below details all 22 test cases created in BranchCoverageTest_coupon_validator.py, mapping each test to its targeted function, control structure branch, and boundary condition:

Target Function	Test Case Name	Branch Evaluated	Test Input / Expected Behavior
coupon_discount	test_cart_total_zero_raises_error	cart_total <= 0 (True)	cart_total=0 -> raises ValueError('empty cart')
coupon_discount	test_cart_total_negative_raises_error	cart_total <= 0 (True)	cart_total=-100 -> raises ValueError('empty cart')
coupon_discount	test_welcome10_new_user_true	WELCOME10 & is_new_user (T)	code='WELCOME10', new_user=True -> 10% discount
coupon_discount	test_welcome10_new_user_false	WELCOME10 & is_new_user (F)	code='WELCOME10', new_user=False -> returns 0
coupon_discount	test_weekend20_saturday_above_500	WEEKEND20, sat, cart>=500 (T)	day='sat', cart=600 -> returns 20% (120)
coupon_discount	test_weekend20_sunday_cart_500	WEEKEND20, sun, cart>=500 (T)	day='sun', cart=500 -> boundary test, returns 100
coupon_discount	test_weekend20_sunday_below_500	WEEKEND20, sun, cart>=500 (F)	day='sun', cart=400 -> returns flat 50
coupon_discount	test_weekend20_weekday	WEEKEND20, day in ('sat','sun') (F)	day='wed' -> weekday check fails, returns 0
coupon_discount	test_flat100_cart_above_300	FLAT100, cart_total > 300 (T)	cart=301 -> returns 100
coupon_discount	test_flat100_cart_exactly_300	FLAT100, cart_total > 300 (F)	cart=300 -> boundary condition, returns 0
coupon_discount	test_flat100_cart_below_300	FLAT100, cart_total > 300 (F)	cart=200 -> below threshold, returns 0
coupon_discount	test_unrecognized_coupon_code	Code matching fall-through	code='INVALID_CODE' -> returns 0
best_discount	test_best_discount_empty_codes	for code in codes (0 iter)	codes=[] -> returns initial best (0)
best_discount	test_best_discount_single_coupon	1 iteration, best <= cap	codes=['WELCOME10'] on 200 -> returns 20
best_discount	test_best_discount_multiple_coupons	value > best (T and F)	['WEEKEND20','WELCOME10'] on 600 -> branch 25->23 closed!
best_discount	test_best_discount_exceeds_cap	best > cap (True)	cart=100, disc=50, cap=40 -> capped at 40.0
best_discount	test_best_discount_equal_to_cap	best > cap (False, boundary)	cart=125, disc=50, cap=50 -> returns 50.0
is_expired	test_is_expired_days_valid_zero	days_valid <= 0 (True)	days_valid=0 -> returns True immediately
is_expired	test_is_expired_days_valid_neg	days_valid <= 0 (True)	days_valid=-3 -> returns True
is_expired	test_is_expired_past_validity	today - start > valid (True)	today=20, start=5, valid=10 (15>10) -> returns True
is_expired	test_is_expired_on_expiry_day	today - start > valid (False)	today=15, start=5, valid=10 (10>10 False) -> False
is_expired	test_is_expired_within_validity	today - start > valid (False)	today=8, start=5, valid=10 (3>10 False) -> False